

Project AI Copilot Package - Developer Guide

Tài liệu này dành cho Developer mới bắt đầu dùng Project AI trong repository. Mục tiêu là biến bộ `.github` khá nhiều file thành một cách làm việc rõ ràng: đọc code nhanh hơn, lên kế hoạch chắc hơn, triển khai ít lệch kiến trúc hơn và review có bằng chứng.

Phiên bản tài liệu: 2026-05-13

Phạm vi: `.github/copilot-instructions.md`, `.github/copilot`, `.github/prompts`, `.github/agents`, `.github/skills`, `.github/instructions`, `.github/workflows`, `.github/docs`

Mục lục

- Tổng quan nhanh
- Chương 1: Tầm nhìn và sức mạnh
 - 1.1 Vì sao AI Agent/LLM quan trọng với Developer?
 - 1.2 Giá trị thực tế của Project AI
 - 1.3 Kiến trúc mục tiêu
 - 1.4 Luồng dữ liệu cấp cao
 - 1.5 Bản đồ năng lực
- Chương 2: Hướng dẫn sử dụng chi tiết
 - 2.1 Quick start cho Developer mới
 - 2.2 Phân tích codebase
 - 2.3 Trace logic phức tạp
 - 2.4 Workflow thực thi: Plan -> Implement -> Review
 - 2.5 Kết nối Azure DevOps qua MCP
 - 2.6 NotebookLM cho tài liệu nội bộ
 - 2.7 Deep Research cho R&D kiến trúc
 - 2.8 Bảng lệnh thao tác nhanh
 - 2.9 Checklist an toàn cho dự án banking

Tổng quan nhanh

Project AI này không phải là một ứng dụng runtime. Đây là một **AI development operating layer** đặt trong `.github`, dùng để chuẩn hóa cách AI hỗ trợ Developer trong một dự án có yêu cầu kiểm soát cao.

Thành phần	Số lượng	Vai trò
Repository instructions	1	Luật nền cho Copilot/AI khi làm việc trong repo
Copilot playbooks/catalogs	20	Kiến trúc, quy trình, catalog prompt/agent/skill và governance
Prompts	22	Lệnh thao tác nhanh cho phân tích, plan, implement, test, review, R&D
Agents	10	Vai trò chuyên biệt như System Analyst, Planner, Tester, Security Reviewer

Thành phần	Số lượng	Vai trò
Skills	18	Bộ nguyên tắc chuyên sâu cho banking, testing, review, MCP, internal knowledge
Workflow	1	Manual governance checks cho build/test/lint/security/dependency audit
Docs	7	Tài liệu vận hành, changelog, presentation, hướng dẫn sử dụng

Điểm quan trọng nhất: AI trong project này luôn đi theo hướng **reference-first, plan-first, evidence-first**. Nghĩa là AI phải đọc rule và code thật trước, lập kế hoạch trước khi sửa, rồi chứng minh bằng kiểm tra, review và tài liệu.

Chương 1: Tầm nhìn và sức mạnh

1.1 Vì sao AI Agent/LLM quan trọng với Developer?

Với Developer, LLM mạnh nhất không nằm ở việc “viết code thật nhanh”, mà ở khả năng xử lý ngữ cảnh lớn:

- Đọc nhiều file cùng lúc và tóm tắt kiến trúc đang tồn tại.
- Trace một flow đi qua controller, service, repository, database, event, config và test.
- So sánh implementation mới với pattern cũ để tránh tạo ra một kiểu code lạ trong hệ thống.
- Tạo plan có ràng buộc về business rule, blast radius, rollback và kiểm thử.
- Review từng dòng thay đổi theo bảo mật, dữ liệu, logging, exception và backward compatibility.

Nói bình dân: AI Agent là một “đồng đội đọc repo rất nhanh”, nhưng trong dự án banking nó phải bị ràng buộc bằng quy trình. Project AI này chính là bộ ràng buộc đó.

1.2 Giá trị thực tế của Project AI

Tăng tốc đọc code cũ

Khi nhận một module lạ, thay vì hỏi từng file rời rạc, Developer có thể dùng `/scout`, `/analyze-code`, `/explain-code` hoặc `@System Analyst` để AI lập bản đồ entry point, dependency, business rule và test hiện có.

Ví dụ:

```
/analyze-code Hãy phân tích flow tạo hạn mức khách hàng. Tập trung vào entry point, service chính, bảng dữ liệu, validation và test liên quan.
```

Giảm rủi ro implement lệch kiến trúc

Project có skill `code-solution-fit`, `system-analysis`, `banking-grade-engineering`, `database-data-integrity` và prompt `/banking-plan`. Các phần này ép AI phải tìm pattern sẵn có trước khi đề xuất code mới.

Ví dụ:

```
/banking-plan Cần thêm rule kiểm tra trạng thái hồ sơ trước khi cho phép submit.  
Hãy đọc flow hiện tại, liệt kê file bị ảnh hưởng, rủi ro dữ liệu và test cần thêm.
```

Review có bằng chứng

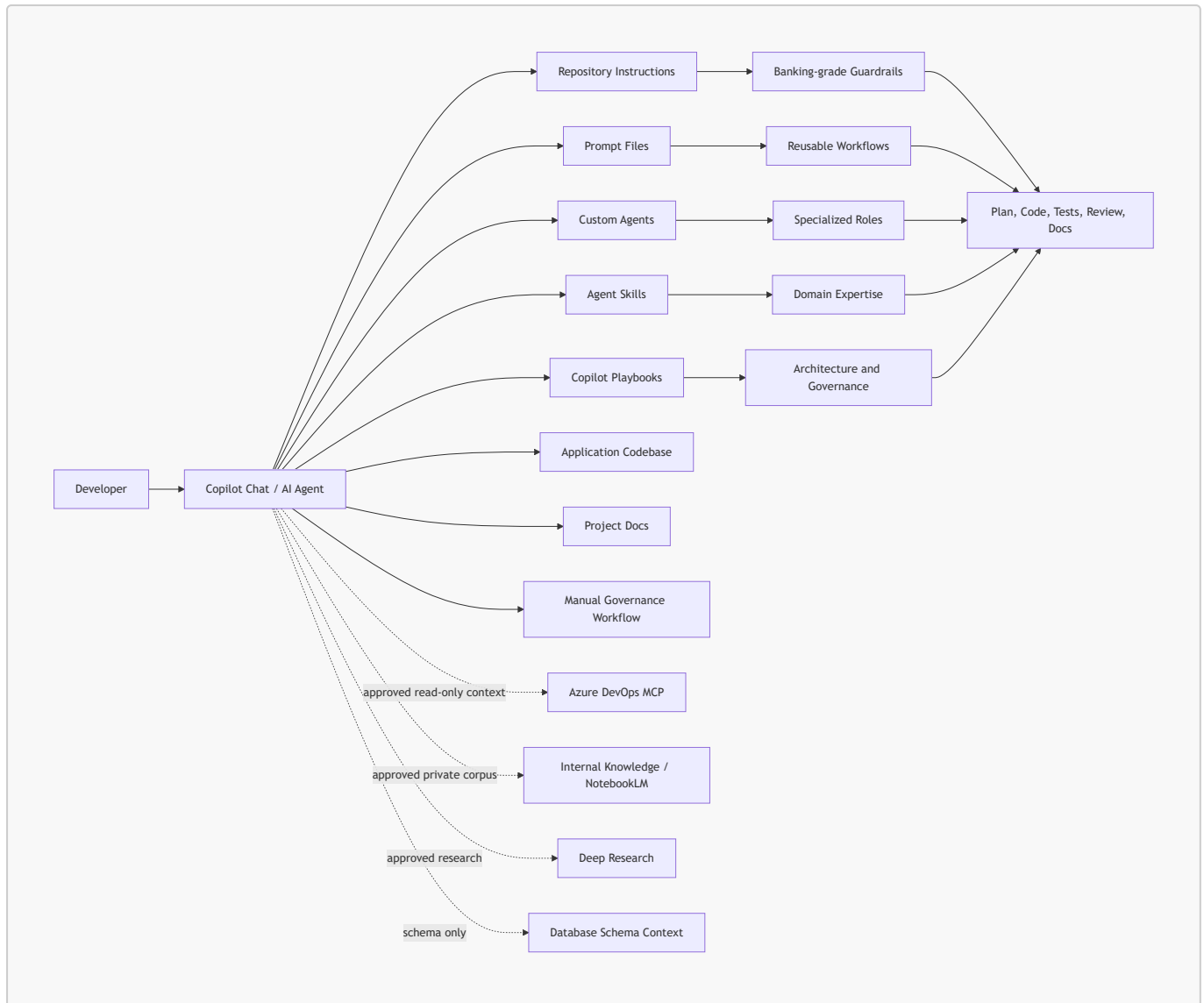
Sau khi sửa, flow chuẩn yêu cầu chạy verification thật của repo, review từng dòng thay đổi và cập nhật tài liệu. Điều này giúp giảm kiểu “AI nói đã ổn” nhưng không có build/test/lint/security đi kèm.

Mở rộng context qua Azure DevOps, tài liệu nội bộ và R&D

Project đã có playbook cho:

- MCP kết nối `dev.azure.com` để đọc work item, repo, wiki, pipeline và search theo quyền tối thiểu.
- Internal knowledge theo kiểu NotebookLM để tra cứu coding convention, tài liệu kỹ thuật và bài học từ dự án cũ.
- Deep Research để so sánh thư viện, kiến trúc và tạo báo cáo R&D có nguồn rõ ràng.

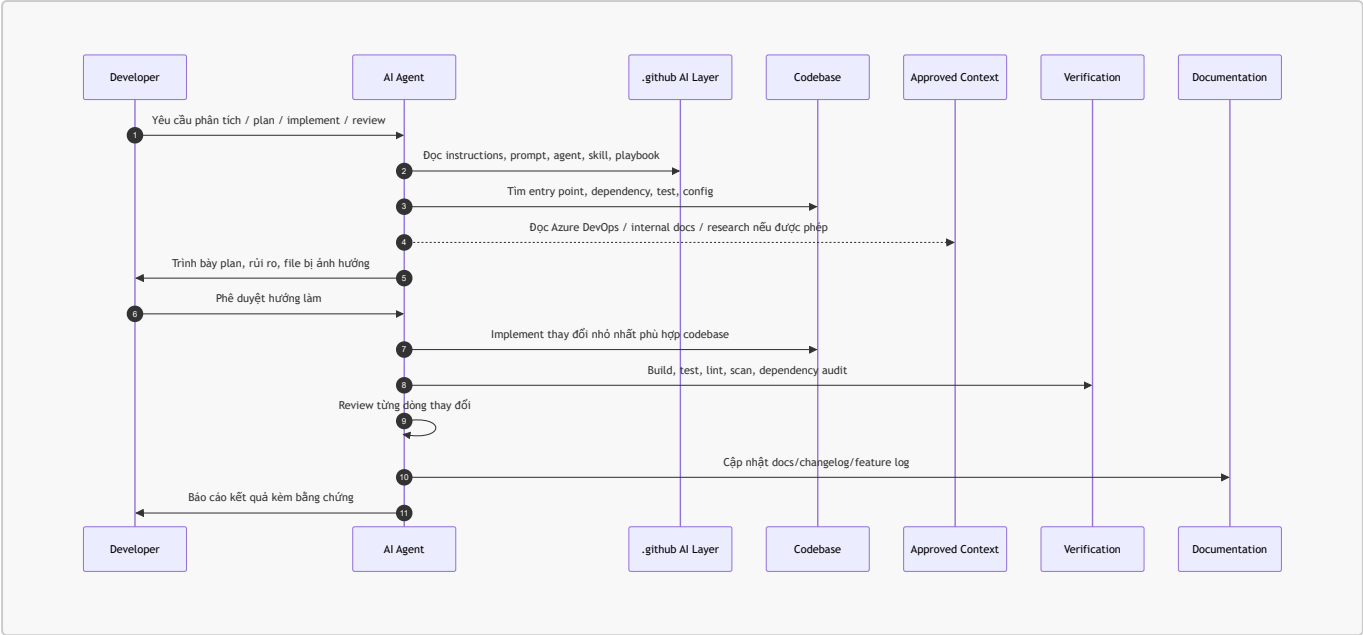
1.3 Kiến trúc mục tiêu



Chú thích:

- **Repository Instructions** là luật nền, luôn được đọc trước.
- **Prompts** là lệnh thao tác nhanh cho từng việc cụ thể.
- **Agents** là vai trò chuyên môn hóa.
- **Skills** là bộ tiêu chuẩn chi tiết cho từng loại nhiệm vụ.
- **Playbooks** là tài liệu quy trình để AI làm việc có kiểm soát.
- **External Context** chỉ dùng theo hướng được phê duyệt, ưu tiên read-only và không đưa secrets vào prompt.

1.4 Luồng dữ liệu cấp cao



1.5 Bản đồ năng lực

Năng lực	File/nhóm chính	Khi nào dùng
Luật nền repository	<code>.github/copilot-instructions.md</code>	Mọi task
Đọc codebase	<code>/scout</code> , <code>/analyze-code</code> , <code>/explain-code</code> , <code>@System Analyst</code> , <code>codebase-reading</code>	Khi vào module lạ hoặc cần hiểu flow
Lên kế hoạch	<code>/plan</code> , <code>/banking-plan</code> , <code>@Planner</code> , <code>planning-governance</code>	Trước khi sửa code
Implement	<code>/implement</code> , <code>code-solution-fit</code> , <code>banking-grade-engineering</code>	Sau khi đã có plan
Debug	<code>/fix</code> , <code>@Debugger</code> , <code>root-cause-debugging</code>	Khi có bug hoặc test fail
Test/verification	<code>/test</code> , <code>@Tester</code> , <code>testing-verification</code> , <code>dotnet-testing</code>	Trước khi bàn giao
Review	<code>/review</code> , <code>/line-review</code> , <code>@Code Reviewer</code> , <code>@Security Reviewer</code>	Sau mỗi thay đổi
Docs	<code>/docs-update</code> , <code>/docs-base-update</code> , <code>@Docs Manager</code>	Khi feature/flow thay đổi
Azure DevOps MCP	<code>/azure-devops-intake</code> , <code>/mcp</code> , <code>mcp-integration-governance</code>	Khi cần đọc work item, wiki, pipeline, repo metadata
Internal knowledge	<code>/internal-knowledge</code> , <code>@Knowledge Curator</code> , <code>internal-knowledge-governance</code>	Khi cần hỏi tài liệu nội bộ hoặc bug cũ
Deep Research	<code>/deep-research</code> , <code>/architecture-research</code> , <code>@Research Architect</code>	Khi cần so sánh thư viện/kiến trúc

Chương 2: Hướng dẫn sử dụng chi tiết

2.1 Quick start cho Developer mới

Mục tiêu của quick start là giúp bạn hiểu repo trong 10-15 phút đầu mà không bị ngợp.

Bước 1: Đọc bản đồ project

```
/scout Hãy tóm tắt cấu trúc repo, entry point chính, test project, config quan trọng và các tài liệu cần đọc trước.
```

Kỳ vọng output:

- Cây thư mục cấp cao.
- Module chính và vai trò của từng module.
- Những file cần đọc trước.
- Vùng rủi ro cao như auth, dữ liệu, transaction, logging, config.

Bước 2: Phân tích flow bạn đang được giao

```
/analyze-code Hãy phân tích flow <tên flow>.  
Liệt kê entry point, lớp xử lý chính, data contract, validation, side effect, test hiện có và điểm cần cẩn thận.
```

Bước 3: Lên plan trước khi sửa

```
/banking-plan Tôi cần thay đổi <mô tả yêu cầu>.  
Hãy đề xuất plan nhỏ nhất, file bị ảnh hưởng, test cần chạy, rủi ro dữ liệu và điều kiện rollback.
```

Bước 4: Implement sau khi plan đã rõ

```
/implement Thực hiện theo plan đã thống nhất.  
Giữ thay đổi nhỏ, bám pattern hiện có, không đổi public contract nếu không cần.
```

Bước 5: Verify và review

```
/test Chạy verification thật của repo và giải thích kết quả.  
/line-review Review từng dòng thay đổi, tập trung vào banking safety, security, dữ
```

```
liệu, logging và backward compatibility.  
/docs-base-update Cập nhật tài liệu dự án dựa trên thay đổi vừa thực hiện.
```

2.2 Phân tích codebase

Khi gặp code cũ, đừng bắt đầu bằng câu “file này làm gì?”. Hãy yêu cầu AI dựng bản đồ từ ngoài vào trong.

Cách hỏi tốt

```
/analyze-code Phân tích module quản lý hồ sơ khách hàng.  
Hãy đi theo thứ tự:  
1. Entry point  
2. Request/response contract  
3. Service orchestration  
4. Repository/database access  
5. Business rules  
6. Error handling/logging  
7. Tests và lỗi hỏng test
```

Cách đọc output

Bạn nên kiểm tra AI có trả lời đủ các phần này không:

- **Entry point:** API, command handler, scheduled job, message consumer hoặc function chính.
- **Layer path:** đường đi qua controller/handler -> service -> repository -> external dependency.
- **Data contract:** DTO, entity, enum, validation rule.
- **Business rule:** điều kiện nghiệp vụ thật, không chỉ mô tả code.
- **Side effect:** ghi DB, publish event, gọi service khác, ghi audit log.
- **Test gap:** chỗ chưa có test hoặc test chưa bao phủ rule quan trọng.

Prompt mẫu cho tóm tắt folder

```
/scout Hãy tóm tắt folder <path>.  
Nêu vai trò từng nhóm file, luồng phụ thuộc, file nào nên đọc đầu tiên và file nào  
có rủi ro cao.
```

2.3 Trace logic phức tạp

Trace logic là việc đi theo một request hoặc event từ lúc vào hệ thống tới khi kết thúc. Với banking, trace nên luôn đi kèm dữ liệu và rule.

Prompt trace flow

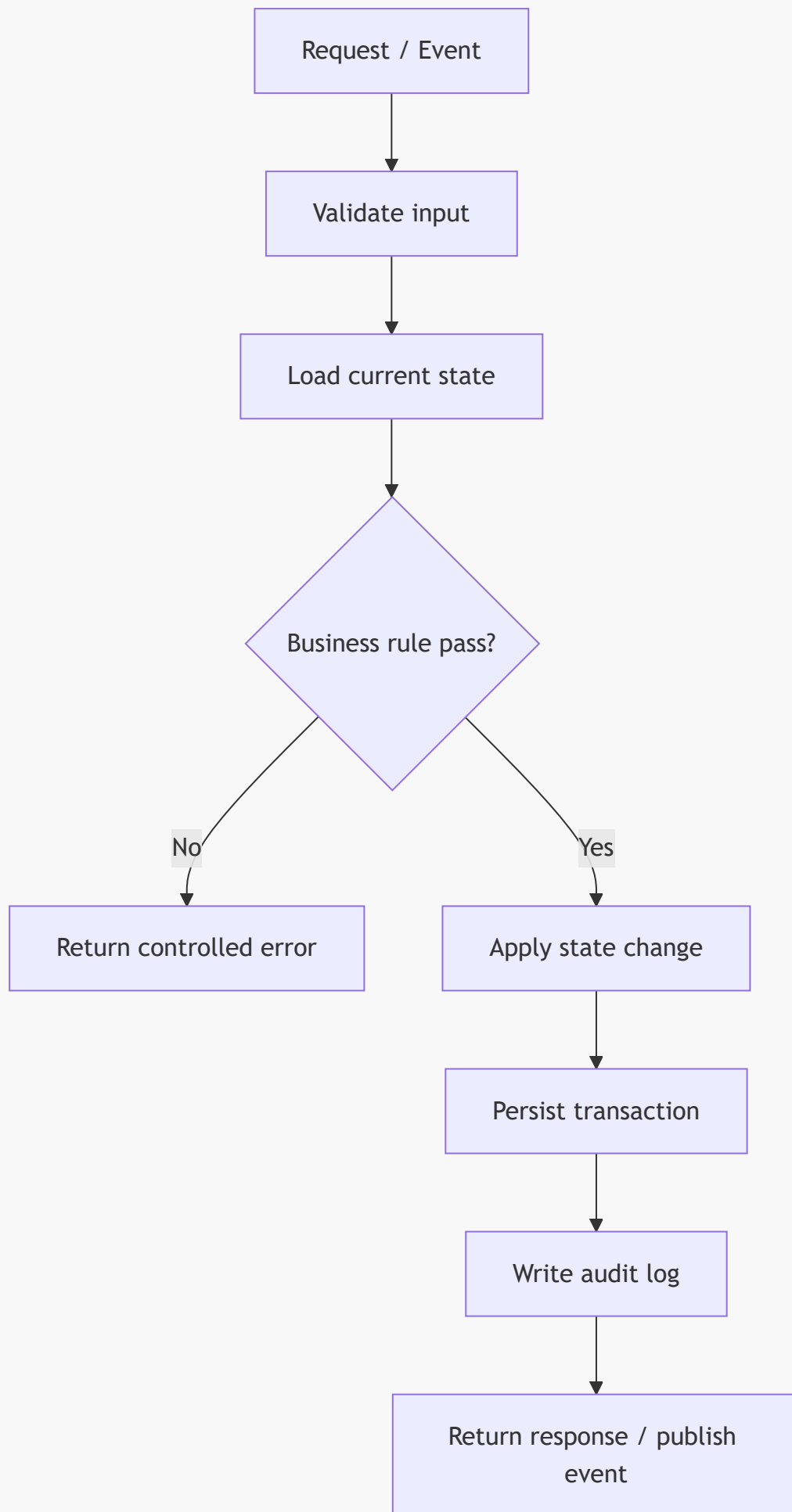
```
/explain-code Trace flow <tên flow hoặc endpoint>.  
Hãy mô tả từng bước theo thứ tự runtime, kèm file/function, dữ liệu đi qua,
```

validation, exception, transaction và audit/log.

Prompt kiểm tra logic

```
/logic-check Kiểm tra logic của flow <tên flow>.  
Tìm nhánh điều kiện thiếu, rule mâu thuẫn, lỗi null/boundary, race condition, sai  
transaction hoặc logging nhạy cảm.
```

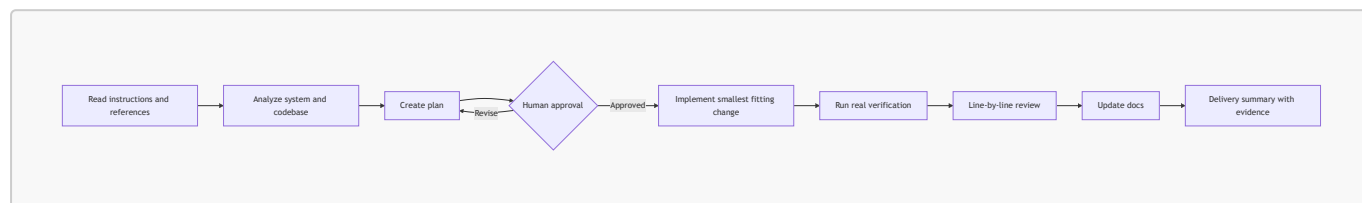
Sơ đồ trace nên yêu cầu AI tạo



Mẹo thực tế: nếu AI bỏ qua database, audit log hoặc transaction, hãy yêu cầu trace lại. Đó thường là nơi bug khó và rủi ro compliance xuất hiện.

2.4 Workflow thực thi: Plan -> Implement -> Review

Workflow chuẩn của project:



Step-by-step

1. Plan

```
/banking-plan <yêu cầu>
```

Yêu cầu AI nêu rõ:

- File sẽ sửa.
- File sẽ không sửa.
- Business rule liên quan.
- Rủi ro dữ liệu.
- Test/verification cụ thể.
- Cần approval ở điểm nào.

2. Implement

```
/implement
```

Làm theo plan đã duyệt.
Không mở rộng scope, không refactor ngoài yêu cầu, không thay đổi contract nếu chưa được phê duyệt.

3. Test

```
/test
```

Chạy verification thật của repo.
Nếu repo có .NET, ưu tiên dotnet restore/build/test/format theo hướng dẫn hiện có.
Nếu không có stack runtime trong repo hiện tại, chạy các command governance phù hợp với .github.

4. Review

```
/line-review Review toàn bộ diff.  
Tập trung vào security, data integrity, exception, logging, audit, performance và  
backward compatibility.
```

5. Docs

```
/docs-base-update Cập nhật docs, changelog và feature-delivery-log cho thay đổi  
vừa thực hiện.
```

2.5 Kết nối Azure DevOps qua MCP

Project này dùng dev.azure.com làm nguồn work item chính.

MCP giúp AI đọc ngữ cảnh công việc từ hệ thống quản lý nội bộ, ví dụ work item, acceptance criteria, wiki kỹ thuật, repository metadata hoặc pipeline status.

Luồng dùng khuyến nghị

```
/azure-devops-intake <Azure DevOps work item URL hoặc ID>  
Hãy đọc requirement, acceptance criteria, linked items, wiki liên quan và đề xuất  
code structure phù hợp với repo hiện tại.
```

Sau đó:

```
/banking-plan Dựa trên work item Azure DevOps vừa đọc, lập plan implement theo  
pattern hiện có.
```

Nguyên tắc an toàn

- Mặc định read-only.
- Chỉ đọc dữ liệu cần cho task.
- Không xuất raw log, secret, token, thông tin khách hàng hoặc dữ liệu nhạy cảm vào prompt.
- Không tự sửa work item, trigger pipeline hoặc thay đổi repo từ MCP nếu chưa được phê duyệt rõ.
- Khi requirement mơ hồ, AI phải ghi assumption và hỏi lại thay vì tự đoán nghiệp vụ.

2.6 NotebookLM cho tài liệu nội bộ

NotebookLM trong bối cảnh project này được hiểu là một kho tri thức riêng tư: coding convention, tài liệu kiến trúc, ADR, incident report, bài học từ dự án cũ và hướng dẫn onboarding.

Khi nào dùng

- Nhân viên mới cần hỏi “module này trước đây thiết kế vì sao?”.

- Cần tìm lại cách xử lý một bug cũ.
- Cần đối chiếu convention nội bộ trước khi viết code.
- Cần hiểu một quyết định kiến trúc đã có trong quá khứ.

Prompt mẫu

```
/internal-knowledge Tìm trong tài liệu nội bộ các convention liên quan đến transaction, audit log và exception handling.  
Tóm tắt rule áp dụng cho task hiện tại, kèm nguồn tài liệu và mức độ chắc chắn.
```

Cách dùng output

AI nên trả lời theo 3 phần:

- **Kết luận áp dụng ngay:** rule nào cần tuân thủ.
- **Nguồn nội bộ:** tài liệu, ADR hoặc guideline nào được dùng.
- **Điểm chưa chắc:** phần nào cần human confirm.

2.7 Deep Research cho R&D kiến trúc

Deep Research dùng khi team cần một báo cáo có nguồn về thư viện, framework hoặc hướng kiến trúc.

Ví dụ phù hợp:

- So sánh monolith modular và microservices cho một domain mới.
- Chọn thư viện background job.
- So sánh performance/cost của các cách cache.
- Đánh giá rủi ro khi nâng version framework.

Prompt mẫu

```
/architecture-research So sánh modular monolith và microservices cho hệ thống xử lý hồ sơ khách hàng.  
Bối cảnh: team nhỏ, yêu cầu audit cao, release theo sprint, cần giảm rủi ro vận hành.  
Xuất báo cáo gồm tiêu chí, bảng so sánh, khuyến nghị, rủi ro và bước thử nghiệm.
```

Output tốt cần có

- Tiêu chí so sánh rõ ràng.
- Nguồn chính thống hoặc đáng tin cậy.
- Bảng trade-off.
- Khuyến nghị gắn với bối cảnh repo.
- Rủi ro và cách kiểm chứng bằng spike/POC.

Lưu ý: Deep Research chỉ tạo khuyến nghị. Quyết định kiến trúc vẫn cần review của Tech Lead/Architect và ghi thành ADR nếu được chấp nhận.

2.8 Bảng lệnh thao tác nhanh

Mục tiêu	Lệnh/prompt	Output mong muốn
Hiểu nhanh repo	<code>/scout</code>	Folder map, entry point, docs/test/config cần đọc
Phân tích flow	<code>/analyze-code</code>	Layer path, business rule, side effect, test gap
Giải thích code	<code>/explain-code</code>	Diễn giải dễ hiểu theo runtime
Kiểm tra logic	<code>/logic-check</code>	Nhánh thiếu, bug boundary, transaction/logging risk
Lập kế hoạch	<code>/plan</code>	Plan kỹ thuật thông thường
Lập kế hoạch banking	<code>/banking-plan</code>	Plan có compliance, data integrity, rollback
Implement	<code>/implement</code>	Code change nhỏ, hợp pattern
Debug	<code>/fix</code>	Root cause, patch, regression test
Test	<code>/test</code>	Verification thật và kết quả
Review	<code>/review</code>	Review cấp PR/change
Review từng dòng	<code>/line-review</code>	Finding theo file/dòng, mức độ rủi ro
Cập nhật docs	<code>/docs-update</code>	Tài liệu feature/flow
Cập nhật docs base	<code>/docs-base-update</code>	Changelog, docs base, delivery log
Đọc Azure DevOps	<code>/azure-devops-intake</code>	Requirement, acceptance criteria, linked context
Dùng MCP	<code>/mcp</code>	Context từ tool được phê duyệt
Hỏi tài liệu nội bộ	<code>/internal-knowledge</code>	Câu trả lời dựa trên private docs
Nghiên cứu chuyên sâu	<code>/deep-research</code>	Báo cáo R&D có nguồn
Nghiên cứu kiến trúc	<code>/architecture-research</code>	So sánh kiến trúc/thư viện và khuyến nghị
Vòng dev đầy đủ	<code>/devloop</code>	Plan -> implement -> verify -> review -> docs

2.9 Checklist an toàn cho dự án banking

Trước khi chấp nhận output của AI, hãy kiểm tra:

- AI đã đọc `.github/copilot-instructions.md` và playbook liên quan chưa?
- AI có nêu file thật, function thật, test thật không?
- Plan có ghi rõ business rule và data integrity không?
- Có thay đổi public API/contract/schema không? Nếu có, đã có approval chưa?
- Có xử lý logging để không lộ dữ liệu nhạy cảm không?
- Có transaction/idempotency/concurrency risk không?
- Có test cho happy path, negative path và boundary không?

- Verification command có chạy thật không?
- Review có chỉ ra file/dòng cụ thể không?
- Docs/changelog/feature log đã cập nhật chưa?

Kết luận

Project AI này là một bộ khung để Developer dùng AI một cách có kiểm soát. Điểm mạnh của nó không chỉ nằm ở prompt hay agent riêng lẻ, mà ở cách kết hợp nhiều lớp: instructions đặt luật, playbook đặt quy trình, prompt tạo thao tác nhanh, agent chia vai, skill thêm chuyên môn, workflow tạo bằng chứng và docs giữ tri thức sống.

Với cách dùng đúng, AI không thay Developer ra quyết định nghiệp vụ. AI giúp Developer nhìn hệ thống nhanh hơn, đặt câu hỏi tốt hơn, triển khai gọn hơn và review chắc hơn.